

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12411836
Kind Code	B2
Date of Patent	September 09, 2025
Inventor(s)	Padhye; Rahul et al.

Ensuring availability and integrity of a database across geographical regions

Abstract

A first stack running on a processor receives the transaction data, reference data, and context data. The reference data is independent of the transaction and of a user. The context data is associated with the user but is independent of the transaction. The first stack strips the transaction of derivable data to obtain stripped data. The derivable data includes data that can be derived from the stripped data, the context data, and the reference data. The derivable data can stream the stripped data to a global database available and redundant across multiple geographical regions. After the first stack fails, a second stack can resume the transaction by retrieving the stripped data from the global database, and retrieving the context data, and the reference data. The second stack can recreate the transaction data based on the stripped data, the context data, and the reference data, and can resume the transaction.

Inventors: Padhye; Rahul (Sammamish, WA), Ganapathy; Anand (Bellevue, WA), Ahmad; Sadique (Bellevue, WA)

Applicant: T-Mobile USA, Inc. (Bellevue, WA)

Family ID: 85177504

Assignee: T-Mobile USA, Inc. (Bellevue, WA)

Appl. No.: 18/746590

Filed: June 18, 2024

Prior Publication Data

Document Identifier	Publication Date
US 20240338356 A1	Oct. 10, 2024

Related U.S. Application Data

Publication Classification

Int. Cl.: G06F16/23 (20190101); G06F16/29 (20190101)

U.S. Cl.:

CPC G06F16/2365 (20190101); G06F16/2379 (20190101); G06F16/29 (20190101);

Field of Classification Search

USPC: None

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
7083092	12/2005	Nakamura	N/A	N/A
7461030	12/2007	Hibler et al.	N/A	N/A
7496541	12/2008	Hibler et al.	N/A	N/A
7496542	12/2008	Hibler et al.	N/A	N/A
7505942	12/2008	Hibler et al.	N/A	N/A
7509293	12/2008	Hibler et al.	N/A	N/A
7542923	12/2008	Waldman	N/A	N/A
7617125	12/2008	Light et al.	N/A	N/A
7788141	12/2009	Sim	N/A	N/A
7797197	12/2009	Kumar et al.	N/A	N/A
7850073	12/2009	Ramachandran et al.	N/A	N/A
7877267	12/2010	Sheppard et al.	N/A	N/A
7890367	12/2010	Ariff et al.	N/A	N/A
8200260	12/2011	Rouse et al.	N/A	N/A
8209229	12/2011	Weiss et al.	N/A	N/A
8219465	12/2011	Aliabadi et al.	N/A	N/A
8332406	12/2011	Donaldson	N/A	N/A
8762258	12/2013	Hansen	N/A	N/A
8781965	12/2013	Huster	N/A	N/A
9412117	12/2015	Hansen	N/A	N/A
9418371	12/2015	Hansen	N/A	N/A
9576296	12/2016	Hansen	N/A	N/A
9584836	12/2016	Fei et al.	N/A	N/A
9600839	12/2016	Calman et al.	N/A	N/A
9620111	12/2016	Bell et al.	N/A	N/A
9754244	12/2016	Hansen	N/A	N/A
9773212	12/2016	Hammad	N/A	N/A
9811810	12/2016	Ranganath et al.	N/A	N/A
9818153	12/2016	Nickel	N/A	N/A

9904913	12/2017	Hansen	N/A	N/A
9911290	12/2017	Zalewski et al.	N/A	N/A
10262307	12/2018	Hansen	N/A	N/A
10332419	12/2018	Prakash et al.	N/A	N/A
10467645	12/2018	Wang et al.	N/A	N/A
10572892	12/2019	Wang et al.	N/A	N/A
10685384	12/2019	Kemp et al.	N/A	N/A
10796276	12/2019	Singh et al.	N/A	N/A
10817893	12/2019	Benjamin	N/A	N/A
10825074	12/2019	Edwards et al.	N/A	N/A
10867317	12/2019	Ladds et al.	N/A	N/A
10885047	12/2020	Mozes et al.	N/A	N/A
10891683	12/2020	Harmon et al.	N/A	N/A
11017403	12/2020	Allbright et al.	N/A	N/A
11055790	12/2020	Unnerstall	N/A	N/A
11188571	12/2020	Chen	N/A	N/A
2003/0208442	12/2002	Cockrill et al.	N/A	N/A
2004/0162740	12/2003	Ericsson et al.	N/A	N/A
2005/0044014	12/2004	Tilis et al.	N/A	N/A
2005/0177507	12/2004	Bandyach et al.	N/A	N/A
2006/0015419	12/2005	Graf et al.	N/A	N/A
2006/0026014	12/2005	Bolduc et al.	N/A	N/A
2007/0150352	12/2006	Kelly-Frank et al.	N/A	N/A
2007/0192222	12/2006	Mika et al.	N/A	N/A
2008/0162590	12/2007	Kundu et al.	N/A	N/A
2008/0167944	12/2007	Keithley et al.	N/A	N/A
2009/0037294	12/2008	Malhotra	N/A	N/A
2009/0281890	12/2008	Aliabadi et al.	N/A	N/A
2010/0057554	12/2009	Lanford	N/A	N/A
2010/0114821	12/2009	Schine	707/623	G06F 16/275
2010/0217688	12/2009	Pandurangan	N/A	N/A
2010/0332307	12/2009	Parento	N/A	N/A
2011/0078011	12/2010	Ariff et al.	N/A	N/A
2011/0093335	12/2010	Fordyce et al.	N/A	N/A
2012/0124496	12/2011	Rose et al.	N/A	N/A
2012/0158600	12/2011	Heyer	N/A	N/A
2012/0166334	12/2011	Kimberg et al.	N/A	N/A
2012/0203632	12/2011	Blum et al.	N/A	N/A
2012/0239481	12/2011	Yankovich et al.	N/A	N/A
2013/0024371	12/2012	Hariramani et al.	N/A	N/A
2013/0054332	12/2012	Ross et al.	N/A	N/A
2013/0060621	12/2012	Walker et al.	N/A	N/A
2014/0278901	12/2013	Hatch et al.	N/A	N/A
2014/0278902	12/2013	Hatch et al.	N/A	N/A
2014/0279190	12/2013	Severinghaus et al.	N/A	N/A
2015/0066687	12/2014	Blackhurst et al.	N/A	N/A
2017/0116599	12/2016	Gupta et al.	N/A	N/A
2017/0132681	12/2016	Modi et al.	N/A	N/A
2017/0148081	12/2016	Chauhan	N/A	N/A
2017/0178166	12/2016	Tiwarly et al.	N/A	N/A

2017/0178219	12/2016	Vaidya	N/A	N/A
2017/0228768	12/2016	Saxena et al.	N/A	N/A
2017/0293982	12/2016	Gupta et al.	N/A	N/A
2018/0101874	12/2017	Laski	N/A	N/A
2019/0012359	12/2018	Wilczynski	N/A	G06F 16/951
2019/0188679	12/2018	Khanwala	N/A	N/A
2019/0197529	12/2018	Nelms et al.	N/A	N/A
2019/0236715	12/2018	Unnerstall	N/A	N/A
2019/0266916	12/2018	Prakash et al.	N/A	N/A
2020/0104865	12/2019	Bhasin	N/A	N/A
2020/0327589	12/2019	Greenberg et al.	N/A	N/A
2021/0019822	12/2020	Edwards et al.	N/A	N/A
2021/0081433	12/2020	Vig et al.	N/A	N/A
2021/0174314	12/2020	Raouda El-Balah	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
1378664	12/2001	CN	N/A
101194257	12/2007	CN	N/A
108027935	12/2017	CN	N/A
111344729	12/2019	CN	N/A
0945812	12/1998	EP	N/A
1285372	12/2002	EP	N/A
1393269	12/2003	EP	N/A
1630732	12/2005	EP	N/A
1736945	12/2005	EP	N/A
2083550	12/2008	EP	N/A
2801065	12/2013	EP	N/A
3292526	12/2017	EP	N/A
3646294	12/2019	EP	N/A
3651108	12/2019	EP	N/A
3776294	12/2020	EP	N/A
2000513854	12/1999	JP	N/A
2008027454	12/2007	JP	N/A
4347508	12/2008	JP	N/A
5054670	12/2011	JP	N/A
2014527252	12/2013	JP	N/A
5771476	12/2014	JP	N/A
6222679	12/2016	JP	N/A
20030068553	12/2002	KR	N/A
100444695	12/2003	KR	N/A
100585306	12/2005	KR	N/A
100644402	12/2005	KR	N/A
20070017244	12/2006	KR	N/A
20070114041	12/2006	KR	N/A
100803942	12/2007	KR	N/A
100804846	12/2007	KR	N/A
20120015430	12/2011	KR	N/A
20140088093	12/2013	KR	N/A
20150092111	12/2014	KR	N/A

9110969	12/1990	WO	N/A
9415309	12/1993	WO	N/A
9641286	12/1995	WO	N/A
9942941	12/1998	WO	N/A
0033221	12/1999	WO	N/A
0060516	12/1999	WO	N/A
0104852	12/2000	WO	N/A
0106439	12/2000	WO	N/A
0113216	12/2000	WO	N/A
0116850	12/2000	WO	N/A
0120514	12/2000	WO	N/A
0154038	12/2000	WO	N/A
0242954	12/2001	WO	N/A
03005149	12/2002	WO	N/A
03021510	12/2002	WO	N/A
03025824	12/2002	WO	N/A
03050741	12/2002	WO	N/A
03054655	12/2002	WO	N/A
03098496	12/2002	WO	N/A
2004042536	12/2003	WO	N/A
2005065346	12/2004	WO	N/A
2006015054	12/2005	WO	N/A
2006055269	12/2005	WO	N/A
2006107776	12/2005	WO	N/A
2006124154	12/2005	WO	N/A
2007026212	12/2006	WO	N/A
2007027641	12/2006	WO	N/A
2007076434	12/2006	WO	N/A
2007086947	12/2006	WO	N/A
2007121298	12/2006	WO	N/A
2008064000	12/2007	WO	N/A
2010028210	12/2009	WO	N/A
2010033081	12/2009	WO	N/A
2010062598	12/2009	WO	N/A
2010127282	12/2009	WO	N/A
2010150229	12/2009	WO	N/A
2011018755	12/2010	WO	N/A
2011031450	12/2010	WO	N/A
2011042615	12/2010	WO	N/A
2011046469	12/2010	WO	N/A
2011049754	12/2010	WO	N/A
2011116297	12/2010	WO	N/A
2012045154	12/2011	WO	N/A
2012054650	12/2011	WO	N/A
2012054786	12/2011	WO	N/A
2012083359	12/2011	WO	N/A
2012087291	12/2011	WO	N/A
2012112822	12/2011	WO	N/A
2012116125	12/2011	WO	N/A
2012118870	12/2011	WO	N/A

2013021287	12/2012	WO	N/A
2013138112	12/2012	WO	N/A
2013155055	12/2012	WO	N/A
2014008471	12/2013	WO	N/A
2014018133	12/2013	WO	N/A
2014059142	12/2013	WO	N/A
2014100588	12/2013	WO	N/A
2014159432	12/2013	WO	N/A
2015112108	12/2014	WO	N/A
2016009168	12/2015	WO	N/A
2016014997	12/2015	WO	N/A
2016032766	12/2015	WO	N/A
2017139042	12/2016	WO	N/A
2018007828	12/2017	WO	N/A
2019118088	12/2018	WO	N/A
2019125611	12/2018	WO	N/A
2020205254	12/2019	WO	N/A

OTHER PUBLICATIONS

Verbitski, Alexandre, et al., “Amazon Aurora: On Avoiding Distributed Consensus for I/Os, Commits, and Membership Changes”, ACM SIGMOD'18, pp. 789-796. (Year: 2018). cited by examiner

Adelberg, Brad, et al., “Database Support for Efficiently Maintaining Derived Data”, ACM 1996, In Proceedings of the 5th International Conference on Extending Database Technology: Advances in Database Technology (EDBT '96), pp. 223-240, year: 1996. cited by applicant

Phatak, Ninad, “Deploying A Modern Data Warehouse With Amazon Redshift”, Amazon Web Services, published Jul. 26, 2021, 60 pages, year: 2021. cited by applicant

Primary Examiner: Waldron; Scott A.

Attorney, Agent or Firm: Perkins Coie LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of U.S. patent application Ser. No. 18/314,350, filed May 9, 2023, entitled “ENSURING AVAILABILITY AND INTEGRITY OF A DATABASE ACROSS GEOGRAPHICAL REGIONS,” which is a continuation of U.S. patent application Ser. No. 17/400,092, filed Aug. 11, 2021, entitled “ENSURING AVAILABILITY AND INTEGRITY OF A DATABASE ACROSS GEOGRAPHICAL REGIONS,” which is hereby incorporated by reference in its entirety.

BACKGROUND

(1) Multi-primary replication is a method of database replication which allows data to be stored by a group of computers and updated by any member of the group. All members are responsive to client data queries. The multi-primary replication system is responsible for propagating the data modifications made by each member to the rest of the group and resolving any conflicts that might arise between concurrent changes made by different members. The problems with multi-primary replication are consistency, performance, and integrity.

(2) Regarding consistency, most multi-primary replication systems are only loosely consistent, i.e.,

lazy and asynchronous, violating ACID (atomicity, consistency, isolation, durability) properties. In computer science, ACID is a set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps. Regarding performance, multi-primary replication systems are complex and increase communication latency. Regarding integrity, issues such as conflict resolution can become intractable as the number of nodes involved rises and latency increases.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) Detailed descriptions of implementations of the present invention will be described and explained through the use of the accompanying drawings.
- (2) FIG. 1 is a block diagram that illustrates a wireless communications system.
- (3) FIG. 2 shows the various data types that the system needs to obtain.
- (4) FIG. 3 shows the flow of dehydrated and rehydrated data.
- (5) FIG. 4 shows the steps performed when a stack is lost.
- (6) FIG. 5 is a flowchart that illustrates a process to ensure availability and integrity of data associated with a transaction.
- (7) FIG. 6 is a block diagram that illustrates an example of a computer system in which at least some operations described herein can be implemented.
- (8) The technologies described herein will become more apparent to those skilled in the art from studying the Detailed Description in conjunction with the drawings. Embodiments or implementations describing aspects of the invention are illustrated by way of example, and the same references can indicate similar elements. While the drawings depict various implementations for the purpose of illustration, those skilled in the art will recognize that alternative implementations can be employed without departing from the principles of the present technologies. Accordingly, while specific implementations are shown in the drawings, the technology is amenable to various modifications.

DETAILED DESCRIPTION

- (9) The disclosed system and method ensure availability and integrity of data associated with a transaction when a stack processing the transaction fails. A stack includes all the layers of code and data the system needs to provide a service to a user, such as completing the transaction. The transactions and associated transaction data can be distributed across multiple geographical regions. The transaction is an interaction between the user and the system which includes transaction data. The transaction can include user inputs, response from a seller, checking of inventory and locations, etc. A stack, e.g., stack A, can operate in a geographical region, such as the Pacific Northwest. Stack A can receive the data associated with the transaction from a user. The data can include an identification (ID) of a purchased item, and a quantity of the purchased item. Stack A can obtain additional data such as reference data and context data. The reference data is independent of the transaction and is independent of the user. The reference data can include tax code, a pricing category associated with the purchased item, and/or product specification associated with the purchased item. The context data is associated with the user and is independent of the transaction. The context data can include user billing information, a user score, a zip code associated with the user, and/or a number of subscriptions (ex. cellular lines, mobile internet or media services) associated with the user's account.
- (10) Stack A can dehydrate the transaction by stripping the transaction from derivable data to obtain dehydrated data. The derivable data can include data derivable from the dehydrated data, the context data, and the reference data, such as a price associated with the purchased item. The dehydrated data can include a purchased item, and a quantity of the purchased item. The derivable

data can include a price associated with the purchased item. Stack A can stream the dehydrated data to a global database. The global database is available and redundant across the multiple geographical regions including the geographical region where the stack A is operating, such as the Pacific Northwest, and a different geographical region, such as the Midwest. The global database is available to stack A and stack B operating independently of stack A. Stack B can operate outside of the Pacific Northwest, such as the Pacific Southwest.

(11) The system can obtain an indication that the stack A has failed. As a result, the system can transfer the transaction to stack B. To resume the transaction, stack B can retrieve the dehydrated data from the global database, and retrieve the context data and the reference data from their respective databases. Stack B can hydrate the dehydrated data by recreating the data associated with the transaction based on the dehydrated data, the context data, and the reference data. After recreating the data, stack B can resume the transaction, without any of data provided by the user.

(12) The description and associated drawings are illustrative examples and are not to be construed as limiting. This disclosure provides certain details for a thorough understanding and enabling description of these examples. One skilled in the relevant technology will understand, however, that the invention can be practiced without many of these details. Likewise, one skilled in the relevant technology will understand that the invention can include well-known structures or features that are not shown or described in detail, to avoid unnecessarily obscuring the descriptions of examples.

(13) Ensuring Availability and Integrity of a Database Across Geographical Regions

(14) FIG. 1 shows multiple stacks interacting with a global database. The system **100** includes a gateway **110**, multiple stacks **120**, **130**, **140**, a streaming service **190**, and a global database **150**. The system **100** can be a part of a wireless telecommunication network providing cellular and data services to the user.

(15) A stack includes all the layers of code **120-1**, **120-2** (only two labeled for brevity) and data **120-3** that the system **100** needs to provide a service to a user. The layers of code can include the application programmer interface (API) **120-1** and the executable code **120-2**.

(16) A stack **120** can fail, and another stack **130**, **140** may need to recreate the data up until the transaction has failed and to resume the transaction from the point of failure. Traditionally, to achieve that, the stacks **120**, **130**, **140** shared data among each other by duplicating data across databases available to each stack **120-3**, **130-3**, **140-3**. In the disclosed system **100**, stacks **120**, **130**, **140** do not share anything with any other stack **120**, **130**, **140**. The system **100** can spin up a stack **120**, **130**, **140** and can communicate with the stack without any dependency with any other stack.

(17) To recover from failure, the stack **120** continuously streams data, using the streaming service **190**, in dehydrated form to the global database **150**. When the stack **120** fails, any of the remaining functioning stacks **130**, **140** can retrieve the dehydrated data **180** from the global database, rehydrate the data, and resume the transaction, as described in this application. To create the dehydrated data **180**, the system breaks the data down into different categories including reference data, context data, and derivable data, as described in this application.

(18) The streaming service **119** ensures that the data stored in the global database **150** is up-to-date. For example, when the stack **120**, **130**, **140** changes transaction data that is not derivable data, the streaming service **190** streams the new data to the global database **150**.

(19) The gateway **110** is a proxy that sits on top of every stack **120**, **130**, **140** and enables communication between the user and the system **100**. The gateway **110** can be API gateway mirror or router. Every request **160** goes to the gateway **110**. Each request **160** can have an authorization header **170**, that uniquely identifies the customer transaction. Based on the authorization header **170**, the gateway **110** can determine to which stack **120**, **130**, **140** to assign the request **160**. The gateway can make the determination based any algorithm including a simple one such as modulo calculation. For example, if there are three stacks **120**, **130**, **140**, the modulo calculation can be modulo 3. Consequently, each stack can receive one third of the traffic. As long as the header (or

customer identifying transaction) doesn't change, the routing logic doesn't change; therefore, the request **160** always goes to the same stack, e.g., stack **120**. If the authorization header **170** changes, (in other words a new transaction starts), the header of the request **160** is changed. In this case, the gateway **110** can assign the request to a different stack based on the modulo calculation.

(20) As long as long as the authorization header **170** doesn't change and the stack **120** is up, the transaction starts and finishes in stack **120**. If the authorization header **170** doesn't change but the stack **120** goes down, the gateway **110** assigns this authorization header **170** to a new stack, e.g., stack **130**. Upon receiving the authorization header **170**, stack **130** identifies that the authorization header is new to the stack **130**. Consequently, the stack **130** gathers context data, applies the reference data and rehydrates the transaction data, as explained in this application, and resumes the transaction from the rehydrated transaction data. Using dehydration, rehydration, reference data, and context data, the system **100** guarantees that if the transaction works with stack **120**, the transaction continues to work with stack **130**. In other words, the transactional integrity is guaranteed.

(21) The database **150** is a global database across multiple stacks **120**, **130**, **140**. The database **150** can be Amazon Redshift, DynamoDB, MongoDB, etc. The database **150** stores dehydrated data **180** that is available to every stack **120**, **130**, **140**.

(22) The system **100** can adjust the traffic load to each stack **120**, **130**, **140**. For example, the system can send 40% of all traffic to stack **120** and 60% of the traffic to stack **130**. In addition, the system **100** can split the incoming transactions based on a configuration attribute such as consuming application or channel. A channel includes web, retail, care. For example, the web can be split 40/60% and across stacks **120**, **130** but retail traffic can be split 90/10% across stacks **120**, **130**. In another example, the system **100** can dedicate a full stack **120**, **130**, **140** to a particular consumer or sales channel. In a third example, the system can split incoming transactions based on the application from which the transactions are coming.

(23) In a fourth example, the system **100** can route transactions based on whether a stack **120**, **130**, **140** is being tested, also known as the canary release. For example, if stack **120** is being updated and/or tested for a new technology, the system **100** can split the incoming traffic 50/50% between stacks **130**, **140**. Once the stack **120** is ready for testing, the system can split the incoming traffic 10/45/45% between stacks **120**, **130**, **140**, respectively. If the stack **120** is performing well, the system **100** can then adjust the split to 20/40/40%. If the stack **120** is not performing well, the system can adjust the split to 0/50/50%.

(24) FIG. 2 shows the various data types that the system needs to obtain. The data is broken up into three categories: reference data **200**, context data **210**, and transaction data **220** which includes derivable data **230**, and dehydrated data **240**.

(25) The reference data **200** includes information such as store location, user credit, tax codes, pricing information, product catalog information, product specification, promotions, etc. The reference data does not depend on the user performing the transaction and can be obtained from a third-party database **250** ahead of transaction and stored locally in a local database or in a local cache.

(26) The context data **210** includes billing information for a user, user profile information, shopping history, address, zip code, number of subscriber lines, etc. When the user logs in, the system **100** needs the context data **210** including who the user is, where the user is, what kind of subscriptions the user has within the system **100**, etc., so that the system **100** can offer the right products and plans to the user. The context data **210** depends on the user and can change with time. Specifically, the context data **210** can change from interaction to interaction.

(27) The context data **210** can also include the user's shopping history and shopping behavior. For example, if the user is new, has no history with the system **100**, and indicates that they want to buy a new plan, such as a subscription with a wireless telecommunication provider, the system **100** can request the user's zip code to check for availability of coverage in the user's area. The zip code

becomes part of context data **210**.

(28) The system **100** can obtain the context data **210** from a third-party database **260** once the user is identified, such as when the user logs in. The database **260** and the database **250** can be the same database or can be different databases. The system **100** gathers the context data upon identifying the user, so that by the time the user is initiating the transaction, such as beginning to shop, the system **100** has already obtained the needed context data **210**.

(29) The final category of data is data associated with the transaction or transaction data **220** including the derivable data **230** and dehydrated data **240**. The derivable data **230** can include product price and promotions. The dehydrated data **240** can include item identification (ID) in the digital shopping cart ("cart") and how many items in the cart. The transaction data **220** can be cart information, financial transaction information, healthcare transaction information, insurance transaction information, etc. The system **100** calculates the transaction data **220** in real time and/or obtains the transaction data **220** from the current transaction with the user. The system **100** does not obtain transaction data from a database.

(30) The derivable data **230** can change with time based on context data **210** and reference data **200**, such as price. The price and/or promotions can change tomorrow or next month. Consequently, the system calculates the price and/or promotions in real time. The dehydrated data **240** does not change based on context data **210** and reference data **200**. For example, the items in the cart and how many items are in the cart do not change based on context data **210** and/or reference data **200**.

(31) When the system **100** dehydrates the transaction data **220**, the system strips off the information that could change from interaction to interaction, such as pricing and/or promotions. The system **100** keeps the dehydrated data **240** such as the item ID, and quantity of the item. The item can be how many cell phone lines, e.g. plans, the user wants to purchase with the system **100**.

(32) The format of the dehydrated data **240** is independent of the format of the transaction data **220**. For example, the data structure representing the transaction data **220** can change without affecting the data structure representing the dehydrated data **240**. Consequently, if during a canary release, the stack **120** in FIG. 1, under testing, changes the data structure of the transaction data **220**, and the stack **120** fails during testing, another stack **130** in FIG. 1 can retrieve the dehydrated data saved by stack **120** prior to failure, and can interpret the dehydrated data because the dehydrated data has not changed.

(33) Promotion **270**, contained in the transaction data, is also contained in the reference data **200**. Promotions **270** are determined ahead of time and can be downloaded from a third-party database **250**. However, there is a real-time component to calculate promotions **270** which depends on the attributes of the user, that is, context data **210**.

(34) For example, when the user indicates to the system **100** to show the items to the user, the system **100** shows the items that the user is eligible to buy. Further, the system **100** can determine if a promotion **270** is associated with the item. The promotion **270** can contain a qualifying condition that the user needs to satisfy. In a specific example, the promotion **270** can indicate that the user needs to have good credit history. The promotion can indicate that the user needs to have five lines with the wireless telecommunication provider, and if the user has five lines, the sixth line can be discounted. The system **100** consistently performs evaluation of promotions **270** because the promotions depend on context data **210** which can change with time. In a second specific example, during a previous transaction, the user had four lines; however, during the current transaction user has seven lines, showing that the system **100** needs to evaluate promotions constantly.

(35) FIG. 3 shows the flow of dehydrated and rehydrated data. A stack **300** can be the stack **120**, **130**, **140** in FIG. 1. The stack **300** can stream data received from a user into the global database **150**. Prior to streaming the data received from the user, the stack **300** dehydrates the data to obtain dehydrated data **310**. Streaming all the data ensures that the latest changes to the transaction data **220** in FIG. 2 are stored as fast as possible in the second global database **150**, thus reducing the

time window during which the failure of the stack **300** can cause loss of data.

(36) The global database **150** is a key-value pair database. The key-value pair database **150** is available across geographical regions unlike the relational databases **120-3**, **130-3**, **140-3** in FIG. 1. The key-value pair database **150** is available across geographical regions, because the key-value pair database **150** is copied across multiple geographical regions. The copying of the key-value pair database **150** can be done because the key-value pair database **150** can be represented in a single file such as a JSON file. If a catastrophic event occurs in one region disabling all the databases **150** that region, another region has a copy and can provide the necessary data. The relational databases **120-3**, **130-3**, **140-3** are not available across geographical regions because copying of relational databases and ensuring synchronization among multiple copies is difficult due to the nature of the relational database, such as preserving complex, hierarchical relations in the relational database. The key-value pair database **150**, because it is easy to copy across multiple geographical regions, ensures availability, while the relational databases **120-3**, **130-3**, **140-3** ensure integrity of the data.

(37) The global database **150** stores the dehydrated data **310**. The dehydrated data **310** has a unique ID in the database **150**. The unique ID can be based on the user ID such as the user phone number, a channel, and/or a segment. A channel includes web, retail, care. A segment can include one or more channels and describes a relationship between two parties engaged over segment. The segment can be business to customer or business to business.

(38) For example, if the ID is based on a user and a channel, when the user is engaged in shopping through web interface, the ID can include the user phone number and the ID of the channel such as "0001" for the web interface. If the stack carrying the transaction fails, the user can go to a brick-and-mortar store. The brick-and-mortar store can retrieve the information of the failed transaction using the customer ID and the ID of the channel. Consequently, the customer can finish the transaction started in one channel on another channel. To retrieve the information of the failed transaction, the stack **300**, different from the stack carrying the web interface transaction, can obtain dehydrated data **310**. The stack **300** can then rehydrate the data in step **320** and can continue the failed transaction. In a different example, the ID can be based on the user ID, and the stack **300** can retrieve the dehydrated data **310** based on solely the user ID without the channel information.

(39) FIG. 4 shows the steps performed when a stack is lost. If a stack **400** is lost, the stack **410** can take over the transaction. Stacks **400**, **410** can be the stacks **120**, **130**, **140** in FIG. 1. The stack **410** can be in a different geographical region than the stack **400**. Upon taking over the transaction, the stack **410** obtains the reference data **420**. Because the reference data **420** is not user specific, the system **405** can store the reference data in a local storage area **430**, such as a cache or a local database. Further, the stack **410** obtains context data **440** from a third-party database **450**. The context data **440** depends on the user. The stack **410** needs to know the user ID **425** to know which context data to retrieve. The user's authorization token **415** contains the user ID **425** and other identifying information. The authorization token **415** is valid during the user's interaction and is validated upon every request. When the stack **400** fails, the system **405** can send the initial request to the stack **410**. The initial request to the stack **410** contains the authorization token **415**. The stack **410** can extract user ID **425** from the authorization token **415**, identify the user, and use the user ID **425** to obtain the appropriate context data.

(40) Finally, the stack **410** obtains a portion of transaction data **490** from the global database **470** which stores dehydrated data streamed by stacks **400**, **410**. The stack **410** obtains the dehydrated data **480**, which can contain information such as item ID and quantity of items. The stack **410** then rehydrates the dehydrated data **480** using reference data **420** and context data **440**. To rehydrate the dehydrated data **480**, the stack **410** can compute the derivable data **460**, such as price and/or promotions of each item in the cart, and the total price of the cart. Upon rehydrating the dehydrated data **480**, the cart obtains transaction data, e.g., rehydrated data, **490**. Rehydrated data **490** includes pricing information.

(41) The user can continue shopping using rehydrated data **490**. As the user modifies the contents

of rehydrated data **490**, the stack **410** continues to stream the modifications to the global database **470** because streaming reduces a window of time during which a failure of the stack **410** can cause loss of user modifications. If the stack **410** fails, another stack can take over the transaction and perform the steps described above.

(42) FIG. 5 is a flowchart that illustrates a process to ensure availability and integrity of data associated with a transaction. In step **500**, a hardware or a software processor executing the instructions described in this application can be associated with a first stack and/or a second stack. The first stack can operate in one geographical region, and the second stack can operate in a different geographical region. For example, the first stack can operate in Portland, and the second stack can operate in Phoenix. The processor can receive the data associated with the transaction, e.g., transaction data, from a user. Transaction data can include an identification (ID) of an item, and a quantity of the item. The item can be a purchased item.

(43) In step **510**, the processor can obtain reference data. The reference data is independent of the transaction, and the reference data is independent of the user. The reference data includes tax code, a pricing category associated with the item, a product specification associated with the item, and identification of a channel associated with the transaction. The channel associated with this transaction describes the medium through which the transaction is completed. The medium through which the transaction is completed can influence the price. For example, a product sold online can have a different price than a product sold in a brick-and-mortar store.

(44) In step **520**, the processor can obtain context data. The context data is associated with the user, and the context data is independent of the current transaction. The context data can include user billing, a user score, a zip code associated with the user, and a number of cellular lines associated with the user's account.

(45) In step **530**, the processor can dehydrate, e.g., strip, the transaction from derivable data to obtain dehydrated data, e.g., stripped data. The processor can update the transaction data, as the user makes changes, and the processor can strip the transaction data as the updates to the transaction are happening. The derivable data includes data that can be derived from dehydrated data, the context data, and the reference data. The dehydrated data can include a purchased item, and a quantity of the purchased item. The derivable data can include a price associated with the purchased item.

(46) In step **540**, the processor can stream the stripped data to a global database. Streaming of updates to the transaction data can happen with a latency of up to a few hundred milliseconds thus minimizing data lost if the stack doing the streaming fails. The global database is a key-value pair database that is available and redundant across multiple geographical regions. The global database is available to both the first stack and the second stack.

(47) In step **550**, the processor can obtain an indication that the first stack has failed. In step **560**, the processor can resume the transaction using the second stack. The second stack can retrieve the stripped data from the global database, and also retrieve the context data and the reference data. The second stack can rehydrate, e.g., recreate, the transaction data based on the dehydrated data, e.g., stripped data, the context data, and the reference data. In step **570**, upon recreating the transaction, the processor can resume the transaction.

(48) To recreate the transaction data, the processor can retrieve the reference data including a promotion associated with the purchased item. The promotion can include a criterion that needs to be satisfied for the promotion to apply. The processor can retrieve the context data such as a user data including data relevant to the criterion. The processor can retrieve the stripped data including a purchased item and a quantity of the purchased item. The processor can determine a price associated with the purchased item based on the promotion, the user data, the purchased item, and the quantity of the purchased item. The processor can add the determined price to the transaction data.

(49) The processor can use multiple stacks to test a canary release of a stack. A canary release is a

deployment pattern that enables the processor to roll out new code/features to a subset of users as an initial test, by adjusting the number of transactions handled by the stack being tested. The processor can create multiple stacks including the first stack and the second stack. Stacks can be associated with channels or segments. Each stack among the multiple stacks is configured to process a portion of a total number of transactions. The processor can assign a portion of the total number of transactions associated with a third stack among the multiple stacks to zero, meaning that the processor takes the third stack offline. The processor can change the third stack by, for example, upgrading the coding of the third stack or changing the data structure of the transaction data that the third stack uses. Upon changing the third stack, the processor can increase the portion of the total number of transactions processed by the third stack. The increase is positively correlated to a performance of the third stack. If the third stack after the change can handle the transactions, the processor can increase the amount of transactions allocated to the third stack.

(50) The processor can assign incoming transactions to various stacks using a modulo function. The processor can create multiple stacks including the first stack and the second stack. Each stack among the multiple stacks is configured to process a portion of a total number of transactions. The processor can obtain a numerical identifier associated with the transaction among the total number of transactions and can perform a modulo operation based on the numerical identifier. A parameter associated with the modulo operation can be configured to distribute the total number of transactions to each stack according to a portion of the total number of transactions assigned to each stack. For example, if there are 10 stacks, and the transactions are distributed equally among the 10 stacks, the modulo operation can be modulo 10. As long as the numerical identifier does not change, the modulo operation guarantees that the same transaction will go to the same stack.

(51) If one of the 10 stacks fails, the modulo operations can be combined. The processor can obtain the numerical identifier of the transactions that went to the failed stack and can perform modulo 9 (the remaining number of stacks) operation of the numerical identifier to evenly distribute the transactions among the functioning stacks.

(52) The processor can create a unique identifier for the dehydrated data stored in the global database. The processor can obtain an identifier (ID) associated with the user, such as the phone number. The processor can also combine the ID associated with the user with the ID of a channel or a segment. The processor can generate a globally unique ID associated with the stripped data based on the ID associated with the user. Finally, the processor can associate the stripped data in the global database with the globally unique ID.

(53) Computer System

(54) FIG. 6 is a block diagram that illustrates an example of a computer system **600** in which at least some operations described herein can be implemented. As shown, the computer system **600** can include: one or more processors **602**, main memory **606**, non-volatile memory **610**, a network interface device **612**, video display device **618**, an input/output device **620**, a control device **622** (e.g., keyboard and pointing device), a drive unit **624** that includes a storage medium **626**, and a signal generation device **630** that are communicatively connected to a bus **616**. The bus **616** represents one or more physical buses and/or point-to-point connections that are connected by appropriate bridges, adapters, or controllers. Various common components (e.g., cache memory) are omitted from FIG. 6 for brevity. Instead, the computer system **600** is intended to illustrate a hardware device on which components illustrated or described relative to the examples of the Figures and any other components described in this specification can be implemented.

(55) The computer system **600** can take any suitable physical form. For example, the computing system **600** can share a similar architecture as that of a server computer, personal computer (PC), tablet computer, mobile telephone, game console, music player, wearable electronic device, network-connected (“smart”) device (e.g., a television or home assistant device), AR/VR systems (e.g., head-mounted display), or any electronic device capable of executing a set of instructions that specify action(s) to be taken by the computing system **600**. In some implementation, the computer

system **600** can be an embedded computer system, a system-on-chip (SOC), a single-board computer system (SBC) or a distributed system such as a mesh of computer systems, or it can include one or more cloud components in one or more networks. Where appropriate, one or more computer systems **600** can perform operations in real time, near real time, or in batch mode.

(56) The network interface device **612** enables the computing system **600** to mediate data in a network **614** with an entity that is external to the computing system **600** through any communication protocol supported by the computing system **600** and the external entity. Examples of the network interface device **612** include a network adaptor card, a wireless network interface card, a router, an access point, a wireless router, a switch, a multilayer switch, a protocol converter, a gateway, a bridge, a bridge router, a hub, a digital media receiver, and/or a repeater, as well as all wireless elements noted herein.

(57) The memory (e.g., main memory **606**, non-volatile memory **610**, machine-readable medium **626**) can be local, remote, or distributed. Although shown as a single medium, the machine-readable medium **626** can include multiple media (e.g., a centralized/distributed database and/or associated caches and servers) that store one or more sets of instructions **628**. The machine-readable (storage) medium **626** can include any medium that is capable of storing, encoding, or carrying a set of instructions for execution by the computing system **600**. The machine-readable medium **626** can be non-transitory or comprise a non-transitory device. In this context, a non-transitory storage medium can include a device that is tangible, meaning that the device has a concrete physical form, although the device can change its physical state. Thus, for example, non-transitory refers to a device remaining tangible despite this change in state.

(58) Although implementations have been described in the context of fully functioning computing devices, the various examples are capable of being distributed as a program product in a variety of forms. Examples of machine-readable (storage) media, machine-readable media, or computer-readable media include recordable-type media such as volatile and non-volatile memory devices **610**, removable flash memory, hard disk drives, optical discs, and transmission-type media such as digital and analog communication links.

(59) In general, the routines executed to implement examples herein can be implemented as part of an operating system or a specific application, component, program, object, module, or sequence of instructions (collectively referred to as “computer programs”). The computer programs typically comprise one or more instructions (e.g., instructions **604**, **608**, **628**) set at various times in various memory and storage devices in computing device(s). When read and executed by the processor **602**, the instruction(s) cause the computing system **600** to perform operations to execute elements involving the various aspects of the disclosure.

(60) Remarks

(61) The terms “example,” “embodiment,” and “implementation” are used interchangeably. For example, references to “one example” or “an example” in the disclosure can be, but not necessarily are, references to the same implementation; and, such references mean at least one of the implementations. The appearances of the phrase “in one example” are not necessarily all referring to the same example, nor are separate or alternative examples mutually exclusive of other examples. A feature, structure, or characteristic described in connection with an example can be included in another example of the disclosure. Moreover, various features are described which can be exhibited by some examples and not by others. Similarly, various requirements are described which can be requirements for some examples but not other examples.

(62) The terminology used herein should be interpreted in its broadest reasonable manner, even though it is being used in conjunction with certain specific examples of the invention. The terms used in the disclosure generally have their ordinary meanings in the relevant technical art, within the context of the disclosure, and in the specific context where each term is used. A recital of alternative language or synonyms does not exclude the use of other synonyms. Special significance should not be placed upon whether a term is elaborated or discussed herein. The use of highlighting

has no influence on the scope and meaning of a term. Further, it will be appreciated that the same thing can be said in more than one way.

(63) Unless the context clearly requires otherwise, throughout the description and the claims, the words “comprise,” “comprising,” and the like are to be construed in an inclusive sense, as opposed to an exclusive or exhaustive sense; that is to say, in the sense of “including, but not limited to.” As used herein, the terms “connected,” “coupled,” or any variant thereof means any connection or coupling, either direct or indirect, between two or more elements; the coupling or connection between the elements can be physical, logical, or a combination thereof. Additionally, the words “herein,” “above,” “below,” and words of similar import can refer to this application as a whole and not to any particular portions of this application. Where context permits, words in the above Detailed Description using the singular or plural number may also include the plural or singular number respectively. The word “or” in reference to a list of two or more items covers all of the following interpretations of the word: any of the items in the list, all of the items in the list, and any combination of the items in the list. The term “module” refers broadly to software components, firmware components, and/or hardware components.

(64) While specific examples of technology are described above for illustrative purposes, various equivalent modifications are possible within the scope of the invention, as those skilled in the relevant art will recognize. For example, while processes or blocks are presented in a given order, alternative implementations can perform routines having steps, or employ systems having blocks, in a different order, and some processes or blocks may be deleted, moved, added, subdivided, combined, and/or modified to provide alternative or sub-combinations. Each of these processes or blocks can be implemented in a variety of different ways. Also, while processes or blocks are at times shown as being performed in series, these processes or blocks can instead be performed or implemented in parallel, or can be performed at different times. Further, any specific numbers noted herein are only examples such that alternative implementations can employ differing values or ranges.

(65) Details of the disclosed implementations can vary considerably in specific implementations while still being encompassed by the disclosed teachings. As noted above, particular terminology used when describing features or aspects of the invention should not be taken to imply that the terminology is being redefined herein to be restricted to any specific characteristics, features, or aspects of the invention with which that terminology is associated. In general, the terms used in the following claims should not be construed to limit the invention to the specific examples disclosed herein, unless the above Detailed Description explicitly defines such terms. Accordingly, the actual scope of the invention encompasses not only the disclosed examples, but also all equivalent ways of practicing or implementing the invention under the claims. Some alternative implementations can include additional elements to those implementations described above or can include fewer elements.

(66) Any patents and applications and other references noted above, and any that may be listed in accompanying filing papers, are incorporated herein by reference in their entireties, except for any subject matter disclaimers or disavowals, and except to the extent that the incorporated material is inconsistent with the express disclosure herein, in which case the language in this disclosure controls. Aspects of the invention can be modified to employ the systems, functions, and concepts of the various references described above to provide yet further implementations of the invention.

(67) To reduce the number of claims, certain implementations are presented below in certain claim forms, but the applicant contemplates various aspects of an invention in other forms. For example, aspects of a claim can be recited in a means-plus-function form or in other forms, such as being embodied in a computer-readable medium. A claim intended to be interpreted as a means-plus-function claim will use the words “means for.” However, the use of the term “for” in any other context is not intended to invoke a similar interpretation. The applicant reserves the right to pursue such additional claim forms in either this application or in a continuing application.

Claims

1. A system comprising: at least one processor; and at least one memory storing executable instructions that, when executed by the at least one processor, cause the at least one processor to: implement a first stack of a distributed service platform, the first stack being coupled to a database of the distributed service platform that is further coupled to a second stack of the distributed service platform; subsequent to the second stack failing to process a transaction, retrieve, by the first stack, a dehydrated data associated with the transaction from the database, the dehydrated data being stored and updated, during a processing of the transaction by the second stack, in the database via a data stream from the second stack to the database; generate, by the first stack, rehydrated data for the transaction, the rehydrated data comprising the dehydrated data retrieved from the database and derived data that is generated based at least on the dehydrated data; and resume, by the first stack, the processing of the transaction at the first stack using the rehydrated data, the rehydrated data reducing a data loss time window associated with the second stack failing to process the transaction.
2. The system of claim 1, wherein the dehydrated data is retrieved in response to an initial request from a gateway for the distributed service platform, the initial request indicating that the processing of the transaction has failed.
3. The system of claim 2, wherein the initial request includes an authorization token, and wherein the dehydrated data is retrieved from the database using the authorization token included in the initial request.
4. The system of claim 1, wherein the first stack is implemented to service incoming requests received by the distributed service platform from a geographic region uniquely designated for the first stack.
5. The system of claim 1, wherein the first stack is a canary deployment and is associated with an assigned portion of the transactions handled by the distributed service platform that is less than respective assigned portions associated with one or more other stacks.
6. The system of claim 5, wherein the instructions further cause the at least one processor to: increase the assigned portion of the transactions associated with the first stack based on the processing of the transaction successfully resuming at the first stack.
7. The system of claim 1, wherein the processing of the transaction is resumed at the first stack based on the first stack being implemented to handle failures of one or more other stacks resulting from the one or more other stacks comprising a canary deployment.
8. The system of claim 1, wherein the first stack comprises an application program interface (API) layer and an executable code layer.
9. A method comprising: retrieving, by a first stack of a distributed service platform, a dehydrated data associated with a transaction from a database of the distributed service platform, subsequent to a second stack failing to process the transaction, the dehydrated data being stored and updated, during a processing of the transaction by the second stack, in the database via a data stream from the second stack to the database; generating, by the first stack, rehydrated data for the transaction, the rehydrated data comprising the dehydrated data retrieved from the database and derived data that is generated based at least on the dehydrated data; and resuming, by the first stack, the processing of the transaction at the first stack using the rehydrated data, the rehydrated data reducing a data loss time window associated with the second stack failing to process the transaction.
10. The method of claim 9, wherein the first stack retrieves the dehydrated data in response to an initial request from a gateway for the distributed service platform, the initial request indicating that the processing of the transaction has failed.
11. The method of claim 10, wherein the initial request includes an authorization token, and

wherein the dehydrated data is retrieved from the database using the authorization token included in the initial request.

12. The method of claim 9, wherein the first stack is implemented to service incoming requests received by the distributed service platform from a geographic region uniquely designated for the first stack.

13. The method of claim 9, wherein the first stack is a canary deployment and is associated with an assigned portion of the transactions handled by the distributed service platform that is less than respective assigned portions associated with one or more other stacks of the distributed service platform.

14. The method of claim 13, further comprising: increasing the assigned portion of the transactions associated with the first stack based on the processing of the transaction successfully resuming at the first stack.

15. The method of claim 9, wherein the processing of the transaction is resumed at the first stack based on the first stack being implemented to handle failures of one or more other stacks of the distributed service platform resulting from the one or more other stacks comprising a canary deployment.

16. The method of claim 9, wherein the first stack comprises an application program interface (API) layer and an executable code layer.

17. One or more non-transitory machine-readable storage medium storing instructions that, when executed by one or more processors of a system, cause the system to: implement a first stack of a distributed service platform, the first stack being coupled to a database of the distributed service platform that is further coupled to a second stack of the distributed service platform; data associated with transactions being processed by one or more other stacks of the distributed service platform; subsequent to the second stack failing to process a transaction, retrieve, by the first stack, a dehydrated data associated with the transaction from the database, the dehydrated data being stored and updated, during a processing of the transaction by the second stack, in the database via a data stream from the second stack to the database; generate, by the first stack, rehydrated data for the transaction, the rehydrated data comprising the dehydrated data retrieved from the database and derived data that is generated based at least on the dehydrated data; and resume, by the first stack, the processing of the transaction at the first stack using the rehydrated data, the rehydrated data reducing a data loss time window associated with the second stack failing to process the transaction.

18. The one or more non-transitory machine-readable storage medium of claim 17, wherein the dehydrated data is retrieved in response to an initial request from a gateway for the distributed service platform, the initial request indicating that the processing of the transaction has failed and comprising an authorization token, and wherein the dehydrated data is retrieved from the database using the authorization token included in the initial request.

19. The one or more non-transitory machine-readable storage medium of claim 17, wherein the first stack is configured with a canary deployment and is associated with an assigned portion of the transactions handled by the distributed service platform that is less than respective assigned portions associated with one or more other stacks.

20. The one or more non-transitory machine-readable storage medium of claim 19, wherein the system is further caused to: increase the assigned portion of the transactions associated with the first stack based on the processing of the transaction successfully resuming at the first stack.
